# Diagramas UML del Sistema de Mapas

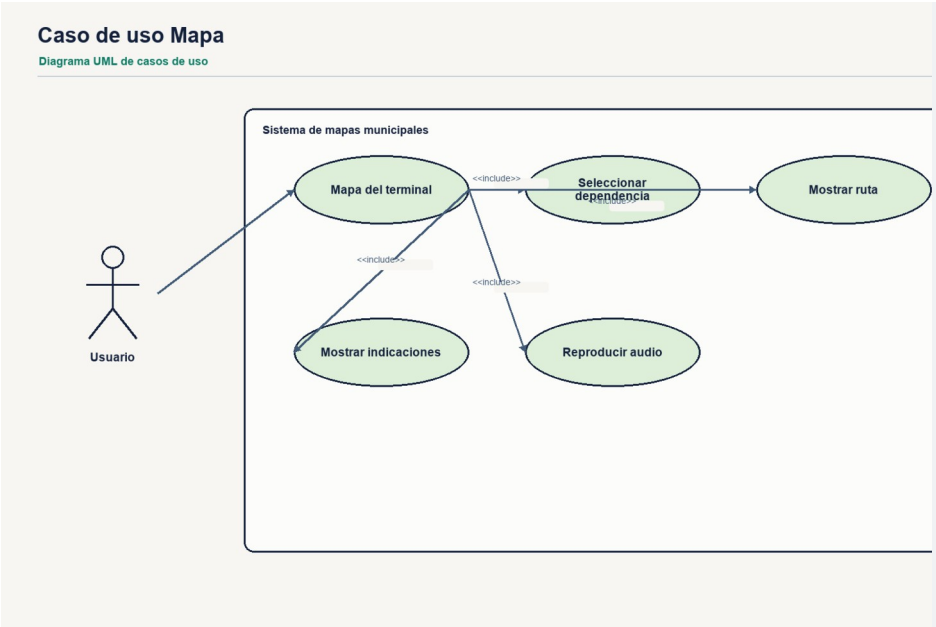
**Mapa  MapaR  MapaConf y MapaRConf**

Cada sección incluye una imagen generada desde PlantUML, la explicación del flujo y el código fuente UML correspondiente. Los archivos individuales se entregan en la carpeta plantuml.

# Casos de uso

## 1. Caso de uso Mapa

Muestra cómo un visitante consulta una dependencia del terminal: selecciona la oficina, observa el recorrido, lee las indicaciones y puede escuchar el audio asociado.




**Codigo PlantUML**

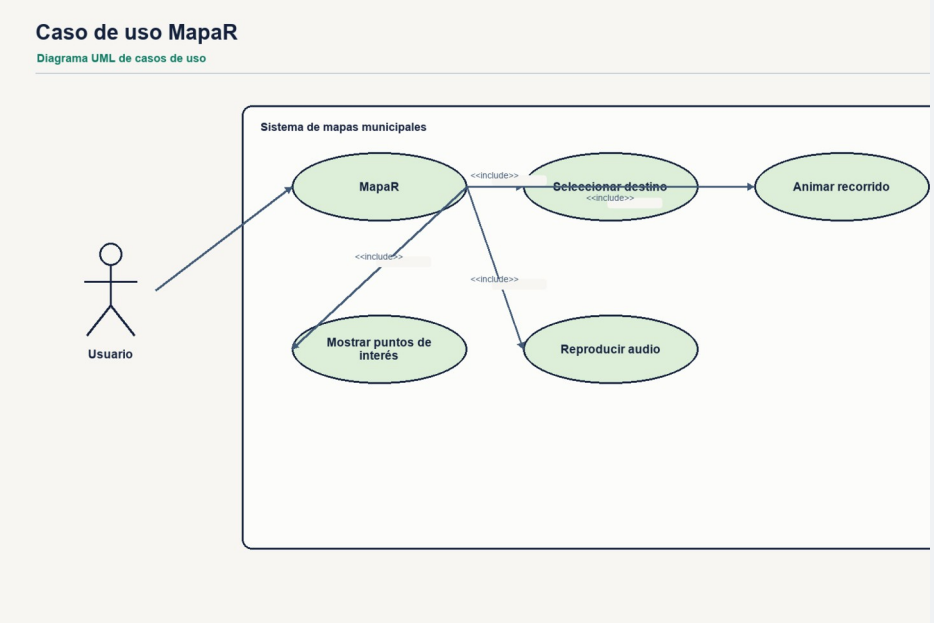
```
@startuml
title Caso de uso Mapa

left to right direction
actor Usuario
rectangle "Mapa del terminal" {
  usecase "Seleccionar
dependencia" as UC1
  usecase "Mostrar ruta" as UC2
  usecase "Mostrar
indicaciones" as UC3
  usecase "Reproducir audio" as UC4
}
Usuario --> UC1
UC1 .> UC2 : <<include>>
UC2 .> UC3 : <<include>>
UC2 .> UC4 : <<extend>>
@enduml
```

*Figura 1. Caso de uso Mapa*

## 2. Caso de uso MapaR

Describe la consulta de un destino urbano. El usuario escoge un destino y el sistema anima el recorrido, presenta puntos de interés, instrucciones y audio.




**Codigo PlantUML**

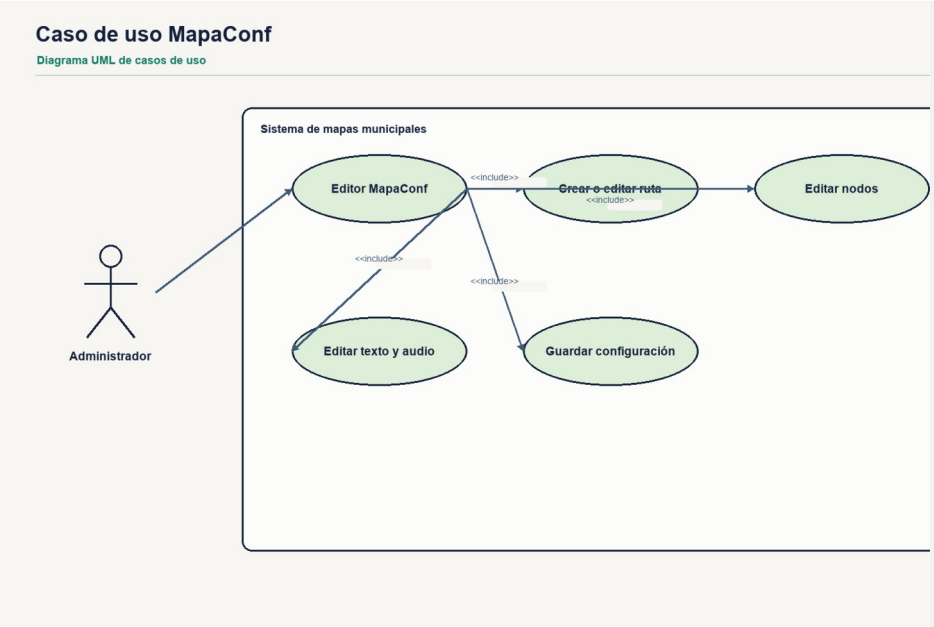
```
@startuml
title Caso de uso MapaR

left to right direction
actor Usuario
rectangle "Mapa urbano de Retalhuleu" {
  usecase "Seleccionar destino" as UC1
  usecase "Animar recorrido" as UC2
  usecase "Mostrar puntos
de interés" as UC3
  usecase "Mostrar
indicaciones" as UC4
  usecase "Reproducir audio" as UC5
}
Usuario --> UC1
UC1 .> UC2 : <<include>>
UC2 .> UC3 : <<include>>
UC2 .> UC4 : <<include>>
UC4 .> UC5 : <<extend>>
@enduml
```

*Figura 2. Caso de uso MapaR*

## 3. Caso de uso MapaConf

Representa las tareas del administrador en el editor del terminal: administrar rutas, definir nodos, elegir assets, editar indicaciones y publicar cambios.




**Codigo PlantUML**

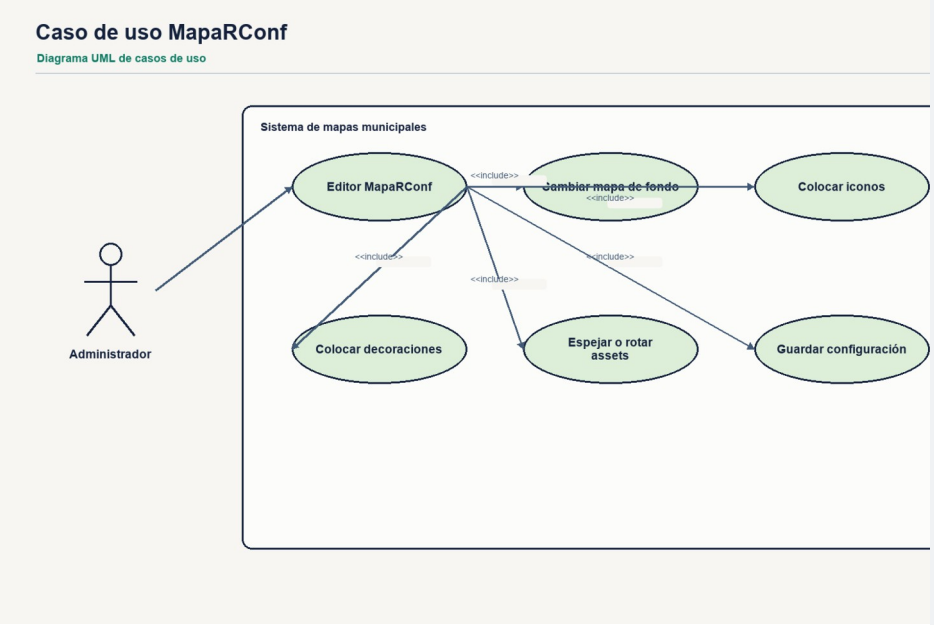
```
@startuml
title Caso de uso MapaConf

left to right direction
actor Administrador
rectangle "Editor MapaConf" {
  usecase "Crear, duplicar o
archivar ruta" as UC1
  usecase "Agregar, mover o
borrar nodos" as UC2
  usecase "Elegir asset
por tramo" as UC3
  usecase "Editar texto
y audio" as UC4
  usecase "Guardar
configuración" as UC5
}
Administrador --> UC1
Administrador --> UC2
Administrador --> UC4
UC1 .> UC3 : <<include>>
UC1 .> UC5 : <<include>>
@enduml
```

*Figura 3. Caso de uso MapaConf*

## 4. Caso de uso MapaRConf

Representa la administración del mapa urbano: rutas, fondo, iconos, decoraciones, transformaciones visuales y persistencia de cambios.




### Codigo PlantUML

```
@startuml
title Caso de uso MapaRConf

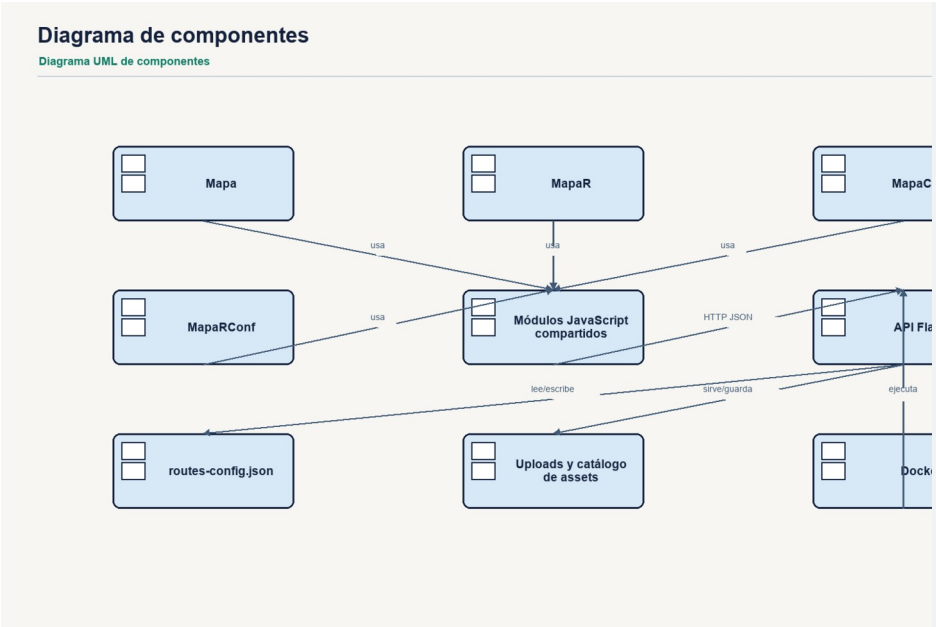
left to right direction
actor Administrador
rectangle "Editor MapaRConf" {
  usecase "Crear o editar
ruta urbana" as UC1
  usecase "Cambiar mapa
de fondo" as UC2
  usecase "Colocar iconos" as UC3
  usecase "Colocar
decoraciones" as UC4
  usecase "Rotar, espejar y
cambiar tamaño" as UC5
  usecase "Guardar
configuración" as UC6
}
Administrador --> UC1
Administrador --> UC2
Administrador --> UC3
Administrador --> UC4
UC3 .> UC5 : <<include>>
UC4 .> UC5 : <<include>>
UC1 .> UC6 : <<include>>
@enduml
```

*Figura 4. Caso de uso MapaRConf*

# Componentes

## 5. Diagrama de componentes

Muestra la arquitectura: cuatro pantallas frontend consumen módulos compartidos y una API Flask, que persiste configuración y sirve los archivos subidos.




**Codigo PlantUML**

```
@startuml
title Diagrama de componentes

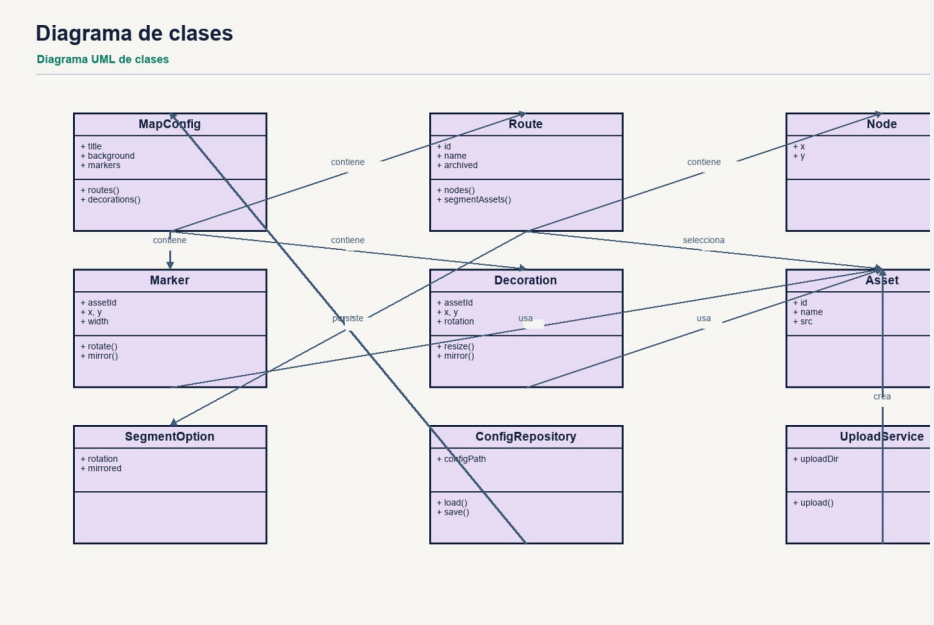
package "Frontend" {
  [Mapa] as mapa
  [MapaR] as mapar
  [MapaConf] as conf
  [MapaRConf] as maparconf
  [Módulos compartidos
config api map utils route viewer] as shared
}
component "API Flask" as api
database "routes config json" as config
folder "Uploads
mapas iconos audios
assets decoraciones" as uploads
node Docker as docker
mapa --> shared
mapar --> shared
conf --> shared
maparconf --> shared
shared --> api : HTTP JSON
api --> config : lee y guarda
api --> uploads : sirve y sube
docker --> api : ejecuta
@enduml
```

*Figura 5. Diagrama de componentes*

# Clases

## 6. Diagrama de clases

Modela la estructura de datos compartida. Un mapa contiene rutas, marcadores y decoraciones; una ruta tiene nodos, assets por tramo y opciones de rotación o espejo.




### Codigo PlantUML

```
@startuml
title Diagrama de clases

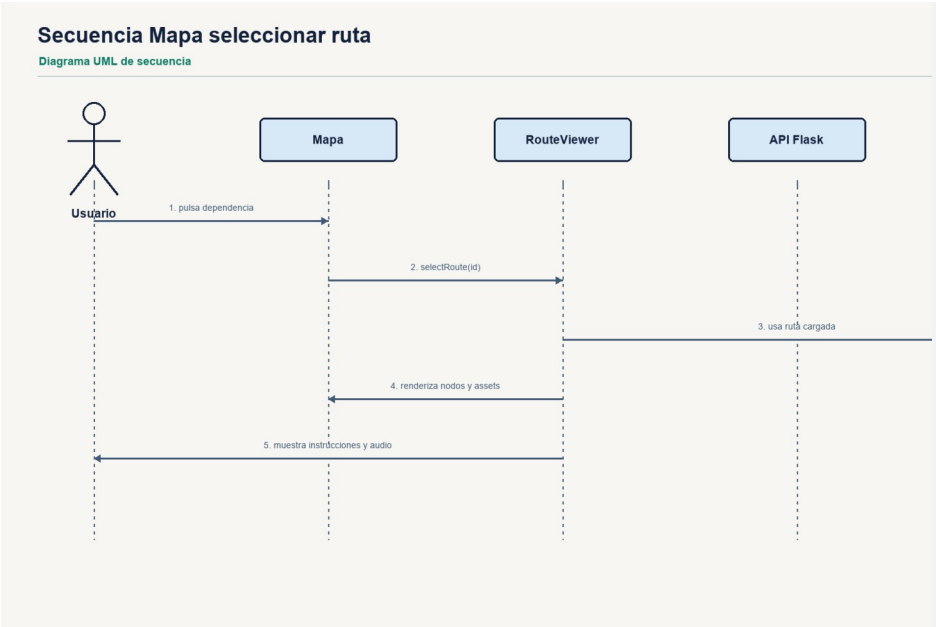
class MapConfig {
  +title: string
  +background: string
  +viewBox: string
}
class Route {
  +id: string
  +name: string
  +archived: boolean
  +instructions: string
  +audio: string
}
class Node { +x: number
+x: number }
class Marker { +assetId: string
+x: number
+y: number
+rotation: number
+mirrored: boolean }
class Decoration { +assetId: string
+x: number
+y: number
+width: number
+rotation: number }
class Asset { +id: string
+name: string
+src: string }
class SegmentOption { +rotation: number
+mirrored: boolean }
class ConfigRepository { +load()
+save() }
class UploadService { +upload(file, kind) }
MapConfig "1" *-- "0..*" Route
MapConfig "1" *-- "0..*" Marker
MapConfig "1" *-- "0..*" Decoration
Route "1" *-- "2..*" Node
Route "1" *-- "0..*" SegmentOption
Route ..> Asset : usa
Marker ..> Asset : usa
Decoration ..> Asset : usa
ConfigRepository --> MapConfig : persiste
UploadService --> Asset : crea
@enduml
```

*Figura 6. Diagrama de clases*

# Secuencias

## 7. Secuencia Mapa seleccionar ruta

Explica el intercambio de mensajes cuando el visitante selecciona una dependencia del terminal y se dibuja el recorrido.




**Codigo PlantUML**

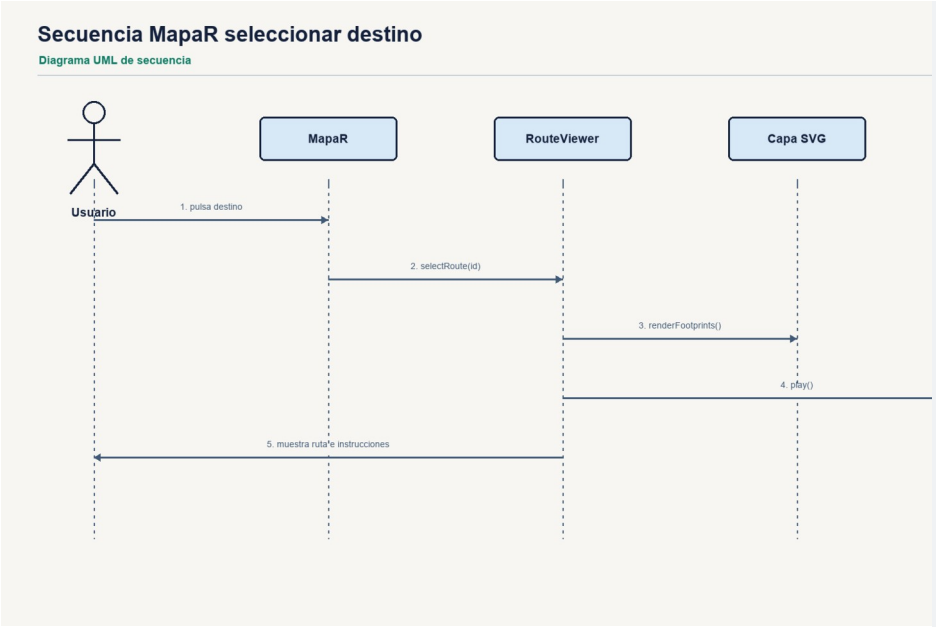
```
@startuml
title Secuencia Mapa seleccionar ruta

actor Usuario
participant Mapa
participant RouteViewer
database "Config JSON" as Config
participant Audio
Usuario -> Mapa : 1. pulsa dependencia
Mapa -> RouteViewer : 2. selectRoute(id)
RouteViewer -> Config : 3. obtiene ruta activa
Config --> RouteViewer : nodos assets instrucciones
RouteViewer -> Mapa : 4. renderiza recorrido SVG
RouteViewer -> Audio : 5. reproduce audio opcional
RouteViewer --> Usuario : 6. muestra pasos
@enduml
```

*Figura 7. Secuencia Mapa seleccionar ruta*

## 8. Secuencia MapaR seleccionar destino

Describe cómo MapaR procesa la selección de un destino, anima los assets y muestra indicaciones para el recorrido urbano.




**Codigo PlantUML**

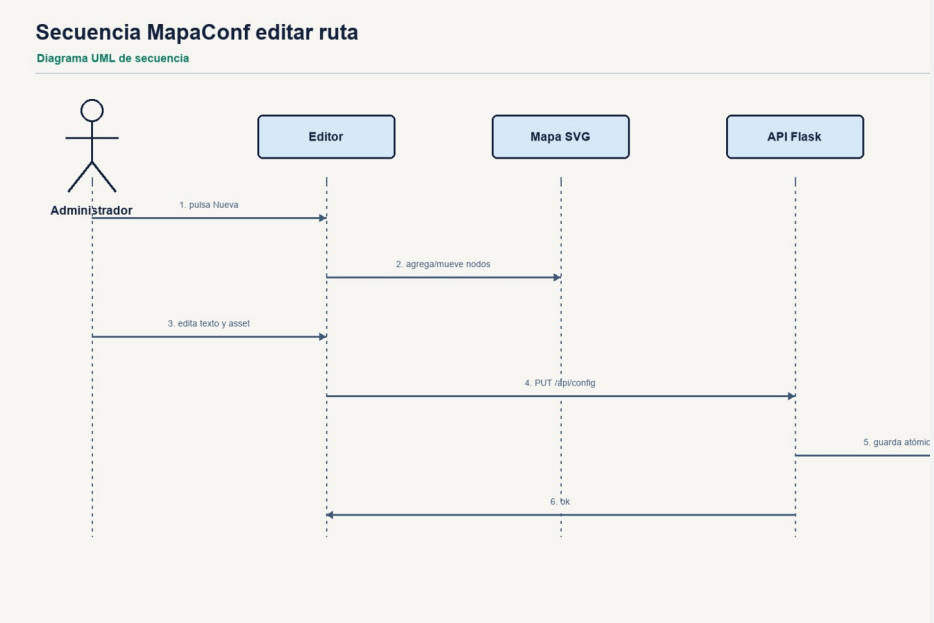
```
@startuml
title Secuencia MapaR seleccionar destino

actor Usuario
participant MapaR
participant RouteViewer
participant "Capa SVG" as SVG
participant Audio
Usuario -> MapaR : 1. pulsa destino
MapaR -> RouteViewer : 2. selectRoute(id)
RouteViewer -> SVG : 3. renderFootprints()
RouteViewer -> SVG : 4. renderMarkers()
RouteViewer -> Audio : 5. play()
RouteViewer --> Usuario : 6. muestra indicaciones
@enduml
```

*Figura 8. Secuencia MapaR seleccionar destino*

## 9. Secuencia MapaConf editar ruta

Muestra el ciclo de edición de una ruta del terminal desde la modificación visual hasta el guardado atómico de la configuración.




**Codigo PlantUML**

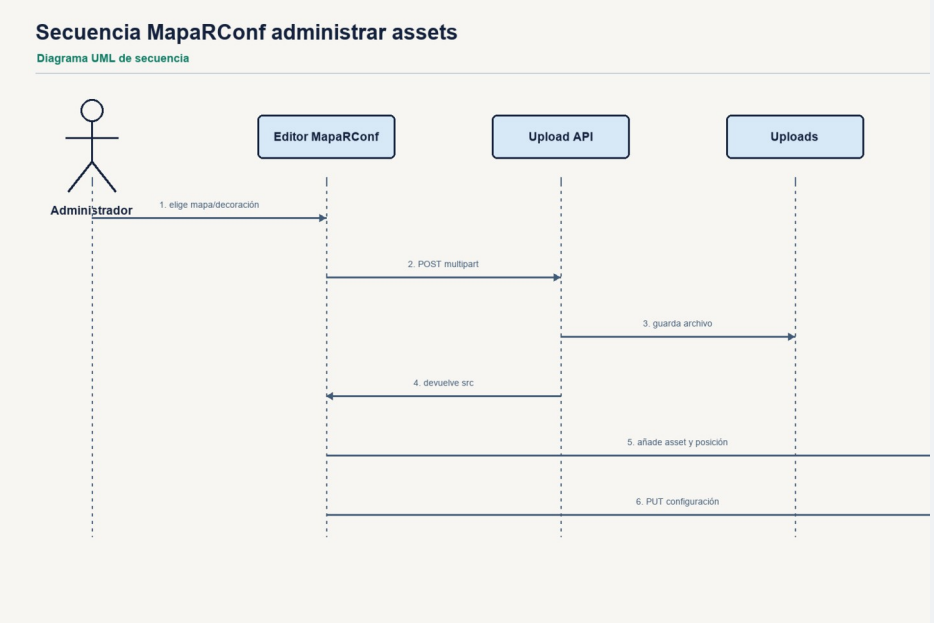
```
@startuml
title Secuencia MapaConf editar ruta

actor Administrador
participant RouteEditor
participant "Mapa SVG" as SVG
participant "API Flask" as API
database "routes config json" as Config
Administrador -> RouteEditor : 1. crea o selecciona ruta
Administrador -> RouteEditor : 2. modifica nodos assets texto
RouteEditor -> SVG : 3. actualiza vista previa
Administrador -> RouteEditor : 4. pulsa Guardar
RouteEditor -> API : 5. PUT api config
API -> Config : 6. guarda archivo temporal
API -> Config : 7. reemplaza config atómicamente
API --> RouteEditor : 8. confirmación
@enduml
```

*Figura 9. Secuencia MapaConf editar ruta*

## 10. Secuencia MapaRConf administrar assets

Explica la subida de un asset o mapa, su registro en el catálogo y la persistencia de su posición dentro del mapa urbano.




**Codigo PlantUML**

```
@startuml
title Secuencia MapaRConf administrar assets

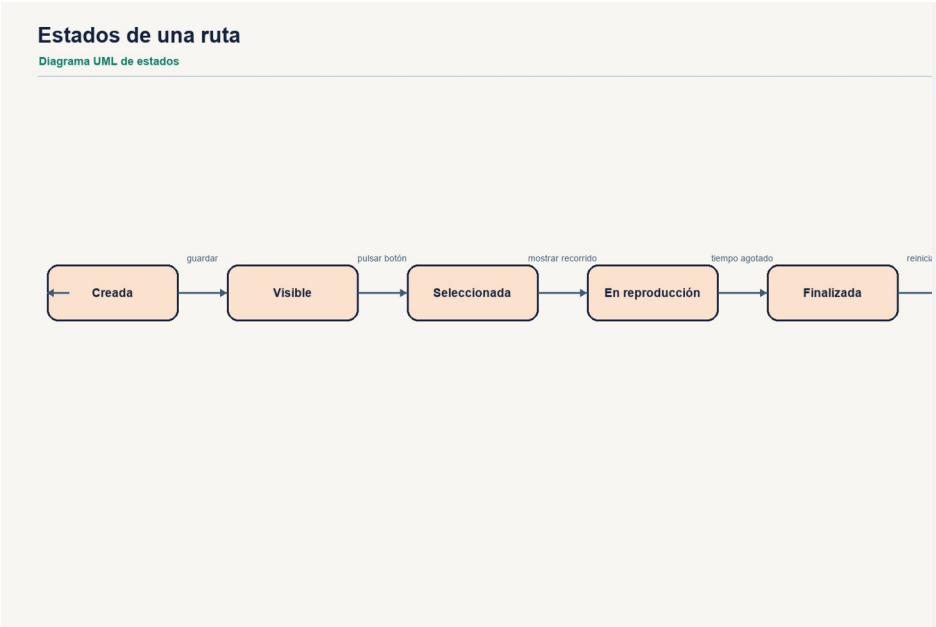
actor Administrador
participant RouteEditor
participant "Upload API" as API
folder Uploads
database "Config JSON" as Config
Administrador -> RouteEditor : 1. selecciona archivo
RouteEditor -> API : 2. POST api upload
API -> Uploads : 3. guarda archivo clasificado
API --> RouteEditor : 4. devuelve src y nombre
Administrador -> RouteEditor : 5. coloca y transforma asset
RouteEditor -> Config : 6. agrega asset y posición
RouteEditor -> API : 7. guarda configuración
@enduml
```

*Figura 10. Secuencia MapaRConf administrar assets*

# Estados

## 11. Estados de una ruta

Explica el ciclo de vida de una ruta desde que se crea hasta que se archiva o vuelve a quedar visible.




**Codigo PlantUML**

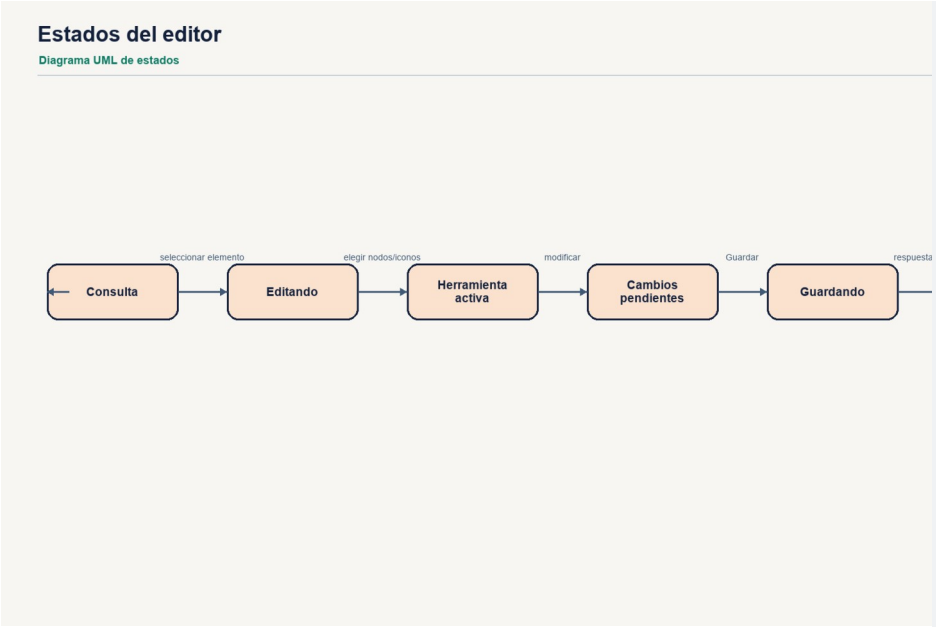
```
@startuml
title Estados de una ruta

[*] --> Creada
Creada --> Visible : guardar
Visible --> Seleccionada : usuario pulsa botón
Seleccionada --> MostrandoRecorrido : renderizar
MostrandoRecorrido --> Visible : temporizador termina
Visible --> Archivada : archivar
Archivada --> Visible : desarchivar
Visible --> [*] : eliminar
@enduml
```

*Figura 11. Estados de una ruta*

## 12. Estados del editor

Muestra los estados de trabajo del editor: selección, edición, cambios pendientes, guardado correcto o error de persistencia.




**Codigo PlantUML**

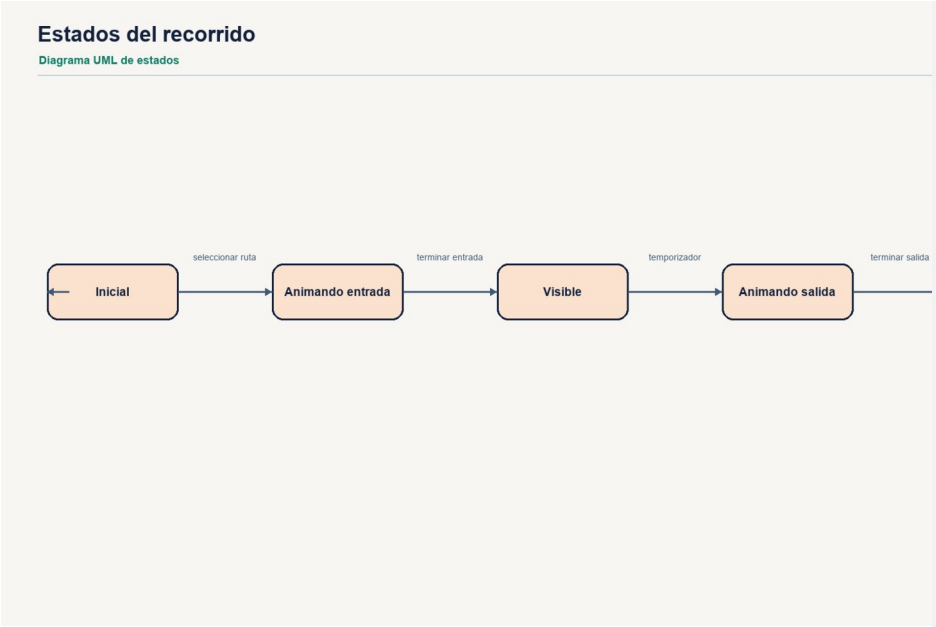
```
@startuml
title Estados del editor

[*] --> Consulta
Consulta --> Edicion : seleccionar ruta
Edicion --> HerramientaActiva : nodos iconos assets
HerramientaActiva --> CambiosPendientes : modificar
CambiosPendientes --> Guardando : Guardar
Guardando --> Guardado : respuesta 200
Guardando --> Error : error API
Error --> CambiosPendientes : corregir
Guardado --> Edicion : continuar
@enduml
```

*Figura 12. Estados del editor*

## 13. Estados del recorrido

Muestra el comportamiento visual del recorrido desde el panel inicial hasta la animación de salida y el reinicio.




**Codigo PlantUML**

```
@startuml
title Estados del recorrido

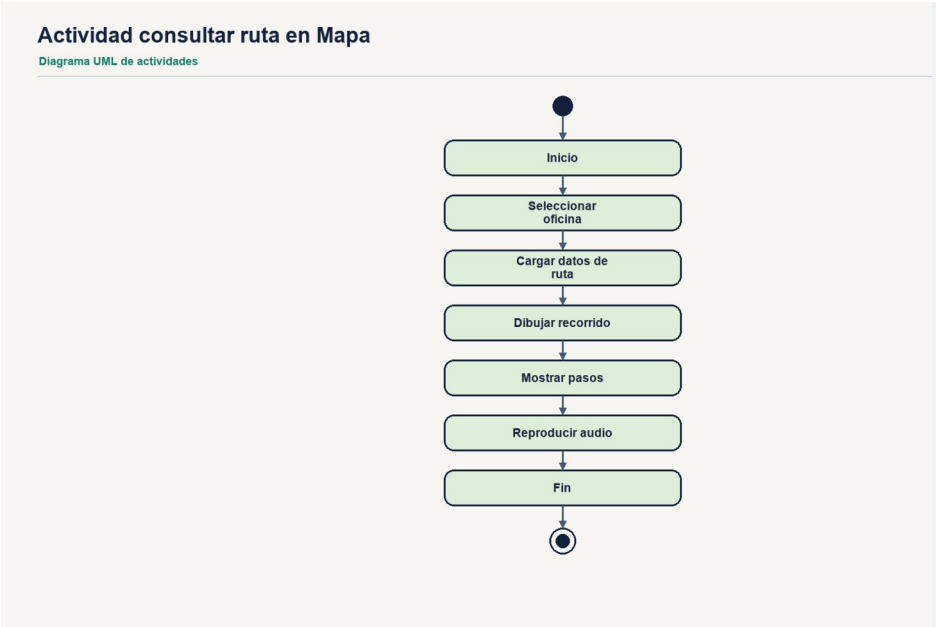
[*] --> PanelInicial
PanelInicial --> Entrada : seleccionar ruta
Entrada --> RutaVisible : animación completa
RutaVisible --> Salida : routeVisibleTime
Salida --> PanelInicial : animación completa
RutaVisible --> Entrada : seleccionar otra ruta
@enduml
```

*Figura 13. Estados del recorrido*

# Actividades

## 14. Actividad consultar ruta en Mapa

Describe el flujo operativo del visor del terminal cuando un visitante consulta una oficina.




**Codigo PlantUML**

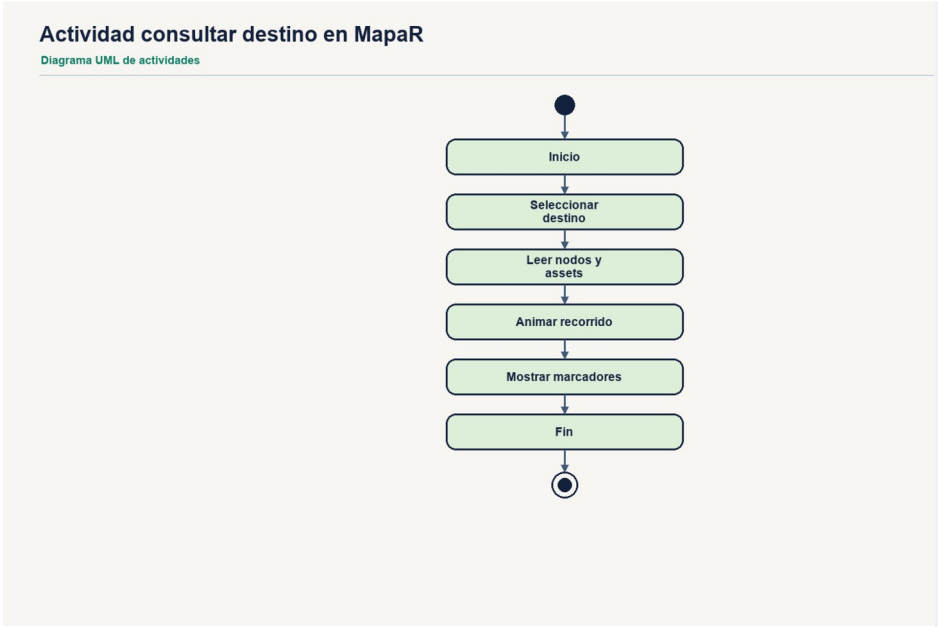
```
@startuml
title Actividad consultar ruta en Mapa

start
:Seleccionar oficina;
:Cargar nodos y assets;
:Dibujar recorrido;
:Mostrar pasos;
if (La ruta tiene audio) then (sí)
  :Reproducir audio;
endif
:Esperar temporizador;
:Ocultar recorrido;
stop
@enduml
```

*Figura 14. Actividad consultar ruta en Mapa*

## 15. Actividad consultar destino en MapaR

Describe la consulta de un destino urbano y la composición del recorrido con marcadores, assets e instrucciones.




**Codigo PlantUML**

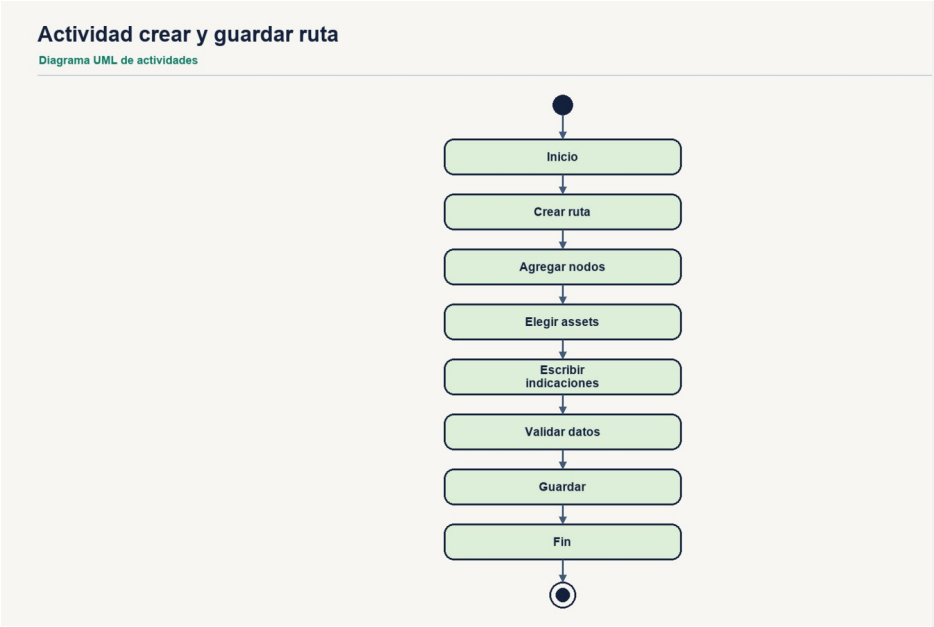
```
@startuml
title Actividad consultar destino en MapaR

start
:Seleccionar destino;
:Leer nodos y segment assets;
:Renderizar huellas o vehículo;
:Mostrar iconos y decoraciones;
:Mostrar indicaciones;
if (Hay audio) then (sí)
  :Reproducir audio;
endif
:Restablecer mapa;
stop
@enduml
```

*Figura 15. Actividad consultar destino en MapaR*

## 16. Actividad crear y guardar ruta

Muestra el proceso que sigue el administrador para publicar una ruta nueva desde cualquiera de los dos editores.




**Codigo PlantUML**

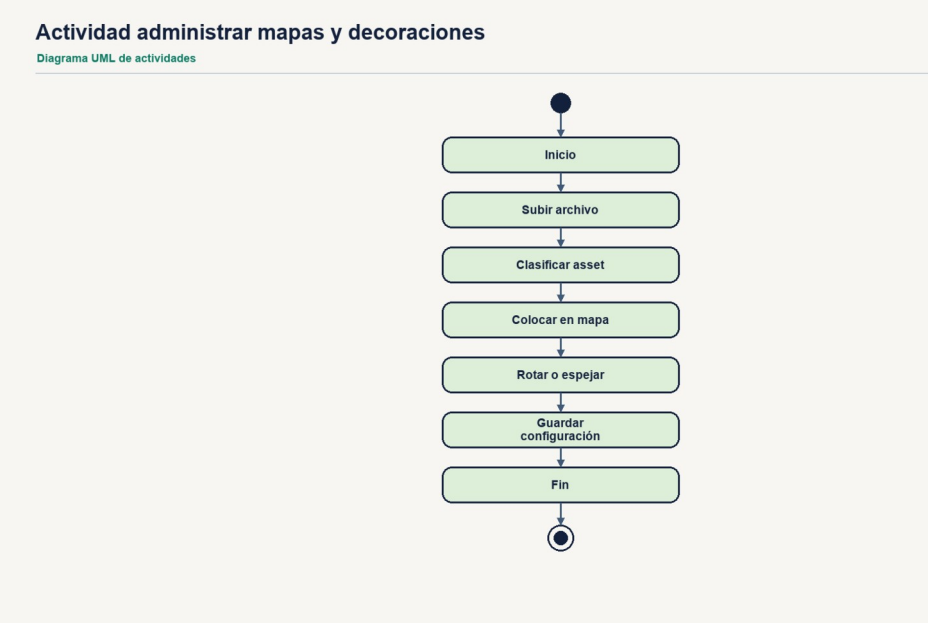
```
@startuml
title Actividad crear y guardar ruta

start
:Crear ruta;
:Definir nombre y color;
:Agregar nodos;
:Elegir assets por tramo;
:Escribir indicaciones;
if (Desea audio) then (sí)
  :Subir o seleccionar audio;
endif
:Guardar configuración;
:Ruta visible en el visor;
stop
@enduml
```

*Figura 16. Actividad crear y guardar ruta*

## 17. Actividad administrar mapas e iconos

Muestra cómo el administrador sube o selecciona un mapa, icono o decoración, aplica transformaciones y guarda el resultado.




**Codigo PlantUML**

```
@startuml
title Actividad administrar mapas e iconos

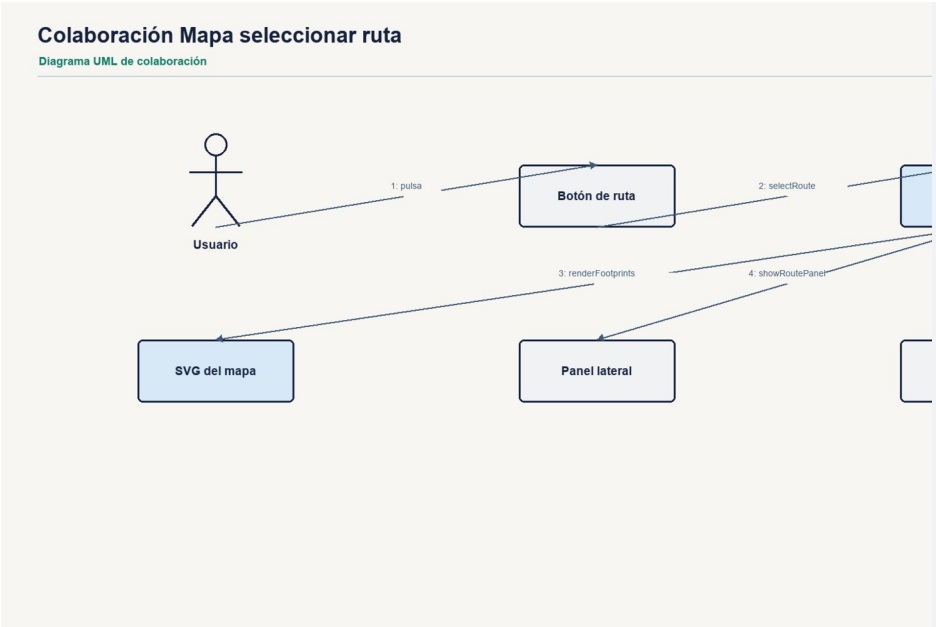
start
:Elegir modo mapa iconos o decoraciones;
:Seleccionar o subir archivo;
:Clasificar asset;
:Colocar sobre el mapa;
:Mover y cambiar tamaño;
:Rotar o espejar;
:Guardar configuración;
stop
@enduml
```

*Figura 17. Actividad administrar mapas e iconos*

# Colaboración

## 18. Colaboración Mapa seleccionar ruta

Muestra los objetos que cooperan durante la selección de una ruta normal, incluyendo interfaz, visor, SVG, panel y audio.




**Codigo PlantUML**

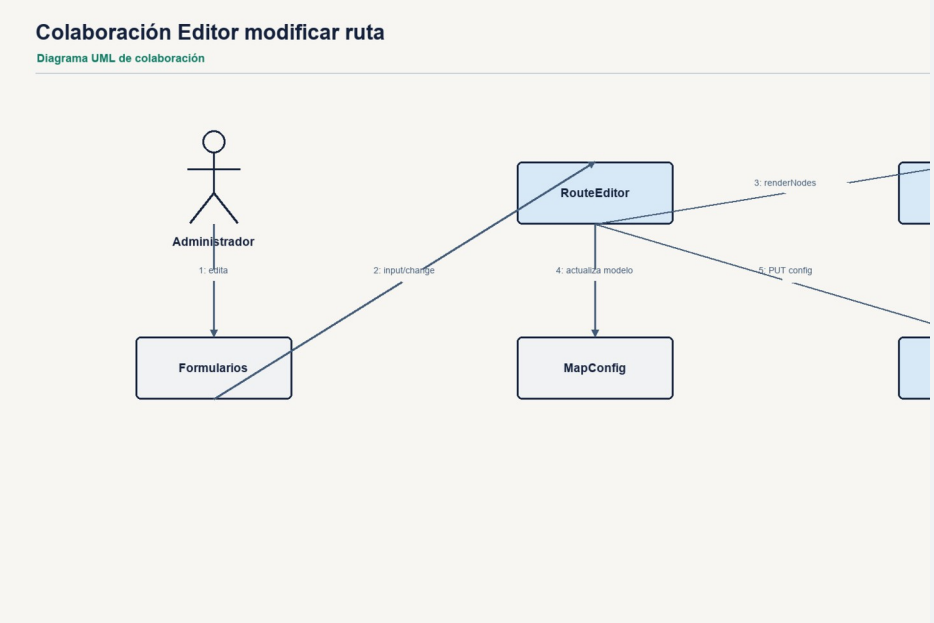
```
@startuml
title Colaboración Mapa seleccionar ruta

object Usuario
object "Botón de ruta" as Boton
object RouteViewer
object "Capa SVG" as SVG
object "Panel lateral" as Panel
object Audio
Usuario --> Boton : 1 pulsa
Boton --> RouteViewer : 2 selectRoute
RouteViewer --> SVG : 3 renderFootprints
RouteViewer --> Panel : 4 showRoutePanel
RouteViewer --> Audio : 5 playRouteAudio
@enduml
```

*Figura 18. Colaboración Mapa seleccionar ruta*

## 19. Colaboración Editor modificar ruta

Muestra la cooperación entre administrador, formularios, editor, modelo de configuración, mapa SVG y la API de guardado.




**Codigo PlantUML**

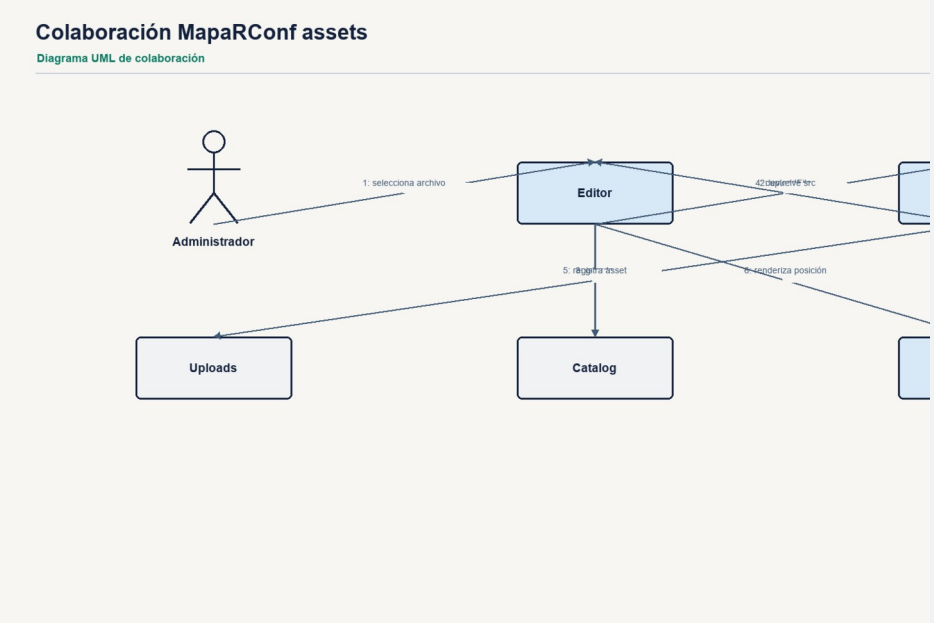
```
@startuml
title Colaboración Editor modificar ruta

object Administrador
object Formularios
object RouteEditor
object "Mapa SVG" as SVG
object MapConfig
object "Save API" as API
Administrador --> Formularios : 1 edita
Formularios --> RouteEditor : 2 input change
RouteEditor --> SVG : 3 renderNodes
RouteEditor --> MapConfig : 4 actualiza modelo
RouteEditor --> API : 5 PUT config
@enduml
```

*Figura 19. Colaboración Editor modificar ruta*

## 20. Colaboración MapaRConf assets

Muestra el intercambio entre editor, API de uploads, catálogo y capa del mapa cuando se sube o coloca un asset.




**Codigo PlantUML**

```
@startuml
title Colaboración MapaRConf assets

object Administrador
object Editor
object "Upload API" as API
object Uploads
object Catalogo
object "Map Layer" as MapLayer
Administrador --> Editor : 1 selecciona archivo
Editor --> API : 2 uploadFile
API --> Uploads : 3 guarda archivo
API --> Editor : 4 devuelve src
Editor --> Catalogo : 5 registra asset
Editor --> MapLayer : 6 renderiza posición
@enduml
```

*Figura 20. Colaboración MapaRConf assets*